

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN



Proyecto final

Programación

Israel Roberto Soto García

CARNET: 9941-25-226

GUATEMALA, 31 MAYO 2026

Evidencia Fotos

```
=====
      SISTEMA DE GESTION DE VENTAS E INVENTARIO
      Desarrollador: Israel Soto Garcia
=====
1. Productos
2. Ventas
3. Reportes
4. Salir
Opcion: █
```

```
1. Registrar
2. Listar
3. Stock
4. Eliminar
5. Volver
Opcion: 2

ID      Nombre      Precio  Stock
1       sal        Q200.00 10016
66      lapiz       Q7.00   99
5       agua pura   Q5.00   51

Presione Enter para continuar...█
```

```
=====
      SISTEMA DE GESTION DE VENTAS E INVENTARIO
      Desarrollador: Israel Soto Garcia
=====
1. Productos
2. Ventas
3. Reportes
4. Salir
Opcion: 2
ID Factura: 8
Dia: 5
Mes: 6
Año: 2026
Codigo producto: 1
Cantidad: 1
Agregado: sal
Agregar otro? (s/n): n
VENTA PROCESADA. TOTAL: Q224.00
```

```
1. Registrar
2. Listar
3. Stock
4. Eliminar
5. Volver
Opcion: 3
Codigo a actualizar: 5
Cantidad (+/-): -
[Error]: Ingrese un numero entero valido.
Cantidad (+/-): +
[Error]: Ingrese un numero entero valido.
Cantidad (+/-): 1
```

```
1. Registrar
2. Listar
3. Stock
4. Eliminar
5. Volver
Opcion: 1
Codigo unico: 7
Nombre: pes
Precio (Q): 99
Stock inicial: 66
Producto registrado.
```

```
1. Registrar
2. Listar
3. Stock
4. Eliminar
5. Volver
Opcion: 4
Codigo a eliminar: 7
Producto dado de baja.

Presione Enter para continuar...
```

```
1. Ventas por Mes
2. Ordenar por Precio
3. Exportar TXT
4. Volver
Opcion:
```

```
1. Ventas por Mes
2. Ordenar por Precio
3. Exportar TXT
4. Volver
Opcion: 1
```

```
RECAUDACION MENSUAL:
May: Q448.00
```

```
Presione Enter para continuar...
```

```
1. Ventas por Mes
2. Ordenar por Precio
3. Exportar TXT
4. Volver
Opcion: 2
CATALOGO ORDENADO POR PRECIO:
agua pura - Q5.00
lapiz - Q7.00
sal - Q200.00
```

```
Presione Enter para continuar...
```

SystemaVentas > - Reporte_inventario.txt

```
1  SISTEMA INVENTARIO - REPORTE
2
3  COD: 1 | sal | Stock: 10016
4  COD: 66 | lapiz | Stock: 99
5  COD: 5 | agua pura | Stock: 51
6  
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAR

```
1. Ventas por Mes
2. Ordenar por Precio
3. Exportar TXT
4. Volver
Opcion: 3
Exportado a 'Reporte_inventario.txt'.

Presione Enter para continuar... 
```

Código de los archivos

main.cpp

```
#include "sistema.h"
```

```
#include <locale>
```

```
int main() {
```

```
    std::setlocale(LC_ALL, "spanish");
```

```
    menuPrincipal();
```

```
    return 0;
```

```
}
```

modelos.h

```
#ifndef MODELOS_H
```

```
#define MODELOS_H
```

```
const int MAX_NOM = 50;
```

```
const int MAX_DETALLES = 10;
```

```
const double IVA_GUATEMALA = 0.12;
```

```
struct Producto {
```

```
    int codigo;
```

```
    char nombre[MAX_NOM];
```

```
    double precio;
```

```
    int stock;
```

```
    int ventasAcumuladas;
```

```
    bool activo;
```

```
};
```

```
struct DetalleVenta {
```

```
    int codigoProducto;
```

```
    int cantidad;
```

```
    double precioUnitario;
```

```
    double subtotal;
```

```
};
```

```
struct Venta {
```

```
int idVenta;  
int dia, mes, anio;  
DetalleVenta detalles[MAX_DETALLES];  
int cantidadDetalles;  
double subtotalGeneral;  
double iva;  
double descuento;  
double total;  
};  
#endif
```


productos.cpp

```
#include "productos.h"

#include "utilidades.h"

#include <iostream>

#include <fstream>

#include <cstring>

const char* FILE_PROD = "productos.dat";

void registrarProducto() {

    std::ofstream archivo(FILE_PROD, std::ios::binary | std::ios::app);

    if (!archivo) return;

    Producto p;

    p.codigo = Util::leerEntero("Codigo unico: ");

    // Validar duplicado

    Producto temp;

    if (obtenerProducto(p.codigo, temp)) {

        std::cout << "Error: El codigo ya existe.\n";

        return;

    }

    Util::leerCadena("Nombre: ", p.nombre, MAX_NOM);

    p.precio = Util::leerDouble("Precio (Q): ");
```

```

    p.stock = Util::leerEntero("Stock inicial: ");
    p.ventasAcumuladas = 0;
    p.activo = true;

    archivo.write(reinterpret_cast<char*>(&p), sizeof(Producto));
    archivo.close();

    std::cout << "Producto registrado.\n";
}

void listarProductos() {
    std::ifstream archivo(FILE_PROD, std::ios::binary);

    Producto p;

    std::cout << "\nID\tNombre\t\tPrecio\tStock\n";

    while (archivo.read(reinterpret_cast<char*>(&p), sizeof(Producto))) {
        if (p.activo) {
            std::cout << p.codigo << "\t" << p.nombre << "\tQ" << p.precio << "\t" << p.stock <<
"\n";
        }
    }

    archivo.close();
}

bool obtenerProducto(int codigo, Producto &p) {
    std::ifstream archivo(FILE_PROD, std::ios::binary);

    if (!archivo) return false;

```

```

while (archivo.read(reinterpret_cast<char*>(&p), sizeof(Producto))) {
    if (p.codigo == codigo && p.activo) {
        archivo.close();

        return true;
    }
}

archivo.close();

return false;
}

void actualizarStock() {
    std::fstream archivo(FILE_PROD, std::ios::binary | std::ios::in | std::ios::out);

    int cod = Util::leerEntero("Codigo a actualizar: ");

    Producto p;

    bool hallado = false;

    while (archivo.read(reinterpret_cast<char*>(&p), sizeof(Producto))) {
        if (p.codigo == cod && p.activo) {
            int mod = Util::leerEntero("Cantidad (+/-): ");

            if (p.stock + mod < 0) {
                std::cout << "Error: Stock insuficiente.\n";
            } else {
                p.stock += mod;

                // Corregir puntero de escritura

                archivo.seekp((long)archivo.tellg() - sizeof(Producto));
            }
        }
    }
}

```

```

        archivo.write(reinterpret_cast<char*>(&p), sizeof(Producto));

        std::cout << "Stock actualizado.\n";

    }

    hallado = true;

    break;

}

}

if (!hallado) std::cout << "No encontrado.\n";

archivo.close();

}

void eliminarProducto() {

    std::fstream archivo(FILE_PROD, std::ios::binary | std::ios::in | std::ios::out);

    int cod = Util::leerEntero("Codigo a eliminar: ");

    Producto p;

    while (archivo.read(reinterpret_cast<char*>(&p), sizeof(Producto))) {

        if (p.codigo == cod && p.activo) {

            p.activo = false;

            archivo.seekp((long)archivo.tellg() - sizeof(Producto));

            archivo.write(reinterpret_cast<char*>(&p), sizeof(Producto));

            std::cout << "Producto dado de baja.\n";

            break;

        }

    }

    archivo.close();

}

```

productos.h

```
#ifndef PRODUCTOS_H
```

```
#define PRODUCTOS_H
```

```
#include "modelos.h"
```

```
void registrarProducto();
```

```
void listarProductos();
```

```
void buscarProductoPorCodigo();
```

```
void buscarProductoPorNombre();
```

```
void actualizarStock();
```

```
void modificarPrecio();
```

```
void eliminarProducto();
```

```
bool obtenerProducto(int codigo, Producto &p);
```

```
#endif
```

reportes.cpp

```
#include "reportes.h"

#include "modelos.h"

#include <iostream>

#include <fstream>

#include <iomanip>

void reportarVentasPorMes() {

    double matriz[12][31] = {0};

    std::ifstream archivo("ventas.dat", std::ios::binary);

    if (!archivo) return;

    Venta v;

    while (archivo.read(reinterpret_cast<char*>(&v), sizeof(Venta))) {

        if (v.mes >= 1 && v.mes <= 12 && v.dia >= 1 && v.dia <= 31)

            matriz[v.mes-1][v.dia-1] += v.total;

    }

    archivo.close();

    std::cout << "\nRECAUDACION MENSUAL:\n";

    const char* meses[] =

{"Ene", "Feb", "Mar", "Abr", "May", "Jun", "Jul", "Ago", "Sep", "Oct", "Nov", "Dic"};

    for (int i = 0; i < 12; i++) {

        double totalMes = 0;

        for (int j = 0; j < 31; j++) totalMes += matriz[i][j];
```

```

        if (totalMes > 0)

            std::cout << meses[i] << ": Q" << std::fixed << std::setprecision(2) << totalMes << "\n";

    }

}

```

```

void ordenarCatálogo() {

```

```

    Producto lista[500];

```

```

    int n = 0;

```

```

    std::ifstream f("productos.dat", std::ios::binary);

```

```

    while (f.read(reinterpret_cast<char*>(&lista[n]), sizeof(Producto))) {

```

```

        if (lista[n].activo) n++;

```

```

    }

```

```

    f.close();

```

```

    for (int i = 0; i < n - 1; i++) {

```

```

        for (int j = 0; j < n - i - 1; j++) {

```

```

            if (lista[j].precio > lista[j+1].precio) {

```

```

                Producto aux = lista[j];

```

```

                lista[j] = lista[j+1];

```

```

                lista[j+1] = aux;

```

```

            }

```

```

        }

```

```

    }

```

```

std::cout << "CATALOGO ORDENADO POR PRECIO:\n";

for (int i = 0; i < n; i++)

    std::cout << lista[i].nombre << " - Q" << lista[i].precio << "\n";
}

void exportarTexto() {

    std::ifstream bin("productos.dat", std::ios::binary);

    std::ofstream txt("Reporte_Inventario.txt");

    Producto p;

    txt << "SISTEMA INVENTARIO - REPORTE\n\n";

    while (bin.read(reinterpret_cast<char*>(&p), sizeof(Producto))) {

        if (p.activo)

            txt << "COD: " << p.codigo << " | " << p.nombre << " | Stock: " << p.stock << "\n";

    }

    bin.close(); txt.close();

    std::cout << "Exportado a 'Reporte_Inventario.txt'.\n";

}

```


reportes.h

```
#ifndef REPORTES_H
#define REPORTES_H

void reportarVentasPorMes();

void ordenarCatálogo();

void exportarTexto();

void estadisticasGlobales();

#endif
```

sistema.cpp

```
#include "sistema.h"
#include "utilidades.h"
#include "productos.h"
#include "ventas.h"
#include "reportes.h"
#include <iostream>

void menuPrincipal() {
    int op;

    do {
        Util::limpiarPantalla();

        Util::mostrarEncabezado();

        std::cout << "1. Productos\n2. Ventas\n3. Reportes\n4. Salir\n";
```

```
    op = Util::leerEntero("Opcion: ");  
    switch(op) {  
        case 1: menuProductos(); break;  
        case 2: crearVenta(); Util::pausar(); break;  
        case 3: menuReportes(); break;  
    }  
} while (op != 4);  
}
```

```
void menuProductos() {  
    int op;  
    do {  
        Util::limpiarPantalla();  
        std::cout << "1. Registrar\n2. Listar\n3. Stock\n4. Eliminar\n5. Volver\n";  
        op = Util::leerEntero("Opcion: ");  
        if (op == 1) registrarProducto();  
        if (op == 2) listarProductos();  
        if (op == 3) actualizarStock();  
        if (op == 4) eliminarProducto();  
        if (op != 5) Util::pausar();  
    } while (op != 5);  
}
```

```
void menuReportes() {  
    int op;
```

```
do {  
    Util::limpiarPantalla();  
    std::cout << "1. Ventas por Mes\n2. Ordenar por Precio\n3. Exportar TXT\n4. Volver\n";  
    op = Util::leerEntero("Opcion: ");  
    if (op == 1) reportarVentasPorMes();  
    if (op == 2) ordenarCatálogo();  
    if (op == 3) exportarTexto();  
    if (op != 4) Util::pausar();  
} while (op != 4);  
}
```

sistema.h

#ifndef SISTEMA_H

#define SISTEMA_H

void menuPrincipal();

void menuProductos();

void menuReportes();

#endif

utilidades.cpp

```
#include "utilidades.h"
```

```
#include <iostream>
```

```
#include <limits>
```

```
#include <stdexcept>
```

```
void Util::mostrarEncabezado() {
```

```
    std::cout << "=====\\n";
```

```
    std::cout << "    SISTEMA DE GESTION DE VENTAS E INVENTARIO    \\n";
```

```
    std::cout << "    Desarrollador: Israel Soto Garcia\\n";
```

```
    std::cout << "=====\\n";
```

```
}
```

```
int Util::leerEntero(const char* mensaje) {
```

```
    int valor;
```

```
    while (true) {
```

```
        std::cout << mensaje;
```

```
        if (std::cin >> valor) return valor;
```

```
        std::cout << "[Error]: Ingrese un numero entero valido.\\n";
```

```
        std::cin.clear();
```

```
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\\n');
```

```
    }
```

```
}
```

```
double Util::leerDouble(const char* mensaje) {
```

```

double valor;

while (true) {

    std::cout << mensaje;

    if (std::cin >> valor && valor >= 0) return valor;

    std::cout << "[Error]: Ingrese un valor decimal positivo.\n";

    std::cin.clear();

    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

}

}

void Util::leerCadena(const char* mensaje, char* destino, int tamano) {

    std::cout << mensaje;

    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

    std::cin.getline(destino, tamano);

}

void Util::limpiarPantalla() {

#ifdef _WIN32

    system("cls");

#else

    system("clear");

#endif

}

void Util::pausar() {

```

```
std::cout << "\nPresione Enter para continuar...";  
std::cin.get();  
}
```

utilidades.h

```
#ifndef UTILIDADES_H
```

```
#define UTILIDADES_H
```

```
namespace Util {
```

```
    void mostrarEncabezado();
```

```
    int leerEntero(const char* mensaje);
```

```
    double leerDouble(const char* mensaje);
```

```
    void leerCadena(const char* mensaje, char* destino, int tamano);
```

```
    void limpiarPantalla();
```

```
    void pausar();
```

```
}
```

```
#endif
```


ventas.cpp

```
#include "ventas.h"
```

```
#include "productos.h"
```

```
#include "utilidades.h"
```

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <cmath>
```

```
void descontarStock(int codigo, int cant) {
```

```
    std::fstream archivo("productos.dat", std::ios::binary | std::ios::in | std::ios::out);
```

```
    Producto p;
```

```
    while (archivo.read(reinterpret_cast<char*>(&p), sizeof(Producto))) {
```

```
        if (p.codigo == codigo) {
```

```
            p.stock -= cant;
```

```
            p.ventasAcumuladas += cant;
```

```
            archivo.seekp((long)archivo.tellg() - sizeof(Producto));
```

```
            archivo.write(reinterpret_cast<char*>(&p), sizeof(Producto));
```

```
            break;
```

```
        }
```

```
    }
```

```
    archivo.close();
```

```
}
```

```
void calcularTotales(Venta &v) {
```

```
    v.subtotalGeneral = 0;
```

```
int items = 0;

for (int i = 0; i < v.cantidadDetalles; i++) {

    v.subtotalGeneral += v.detalles[i].subtotal;

    items += v.detalles[i].cantidad;

}


double desc = 0;

if (v.subtotalGeneral > 500) desc += 0.05;

if (items % 5 == 0) desc += 0.03;


v.descuento = v.subtotalGeneral * desc;

v.iva = (v.subtotalGeneral - v.descuento) * IVA_GUATEMALA;

v.total = (v.subtotalGeneral - v.descuento) + v.iva;

}


void crearVenta() {

    Venta v;

    v.cantidadDetalles = 0;

    v.idVenta = Util::leerEntero("ID Factura: ");

    v.dia = Util::leerEntero("Dia: ");

    v.mes = Util::leerEntero("Mes: ");

    v.anio = Util::leerEntero("Anio: ");


    char opcion;

    do {
```

```

int cod = Util::leerEntero("Codigo producto: ");

Producto p;

if (obtenerProducto(cod, p)) {

    int cant = Util::leerEntero("Cantidad: ");

    if (cant > p.stock) {

        std::cout << "Stock insuficiente (Disponible: " << p.stock << ")\n";

    } else {

        v.detalles[v.cantidadDetalles] = {p.codigo, cant, p.precio, p.precio * cant};

        v.cantidadDetalles++;

        std::cout << "Agregado: " << p.nombre << "\n";

    }

} else {

    std::cout << "Producto no existe.\n";

}

std::cout << "Agregar otro? (s/n): "; std::cin >> opcion;

} while ((opcion == 's' || opcion == 'S') && v.cantidadDetalles < MAX_DETALLES);

if (v.cantidadDetalles > 0) {

    calcularTotales(v);

    std::ofstream f("ventas.dat", std::ios::binary | std::ios::app);

    f.write(reinterpret_cast<char*>(&v), sizeof(Venta));

    f.close();

    for (int i = 0; i < v.cantidadDetalles; i++)

        descontarStock(v.detalles[i].codigoProducto, v.detalles[i].cantidad);

```

```
std::cout << "VENTA PROCESADA. TOTAL: Q" << v.total << "\n";  
}  
}
```

ventas.h

```
#ifndef VENTAS_H
```

```
#define VENTAS_H
```

```
#include "modelos.h"
```

```
void crearVenta();
```

```
void calcularTotales(Venta &v);
```

```
void descontarStock(int codigo, int cant);
```

```
#endif
```